Java 入门笔记

第二课 · 标识符、常量、变量与基本数据类型

学习目标	掌握标识符、变量与常量，理解八种基本数据类型，并能根据数据特点选择合适的类型。

目录

本节目标 2

2.1 Java 语言基础 2

2.2 常量、变量与数据类型 5

综合示例 10

常见易错点 10

课后自测 10

本课小结与下一课衔接 11

整理日期：2026 年 9 月 22 日

> 本课承接 Class 1 的 Java 程序结构：先认识程序中的名称和数据，再为 Class 3 的输入输出、运算符与类型转换打基础。

本节目标

- 能判断标识符是否合法，并区分命名规则与命名规范。
- 理解关键字、标识符和字面量之间的区别。
- 掌握变量的声明、赋值、初始化和作用域。
- 理解 final 常量和各种字面量。
- 掌握 Java 的八种基本数据类型及常见取值范围。
- 能根据数据特点选择合适的类型。

2.1 Java 语言基础

2.1.1 程序中的数据与运算

程序会对数据进行各种处理，例如：

- 算术运算：加、减、乘、除；
- 关系运算：比较大小；
- 逻辑运算：判断真假；
- 其他操作：输入、输出、查询和保存数据。

表达式通常由操作数和运算符组成：

```
int a = 10;
int b = 3;
int sum = a + b;
System.out.println(sum); // 输出 13
```

在表达式 a + b 中，a、b 是操作数，+ 是运算符。运算符的详细规则会在 Class 3 学习。

字符、Unicode 与编码

- 字符：人能够识别的文字或符号，例如 A、□、U+1F600 □□□□。
- Unicode：为字符分配统一码点的标准，例如 □ 的码点是 U+4E2D。
- UTF-8、UTF-16：按照一定规则保存或传输 Unicode 字符的编码方式。
- Java 的一个 char 是 16 位 UTF-16 代码单元，范围为 \u0000～\uFFFF。

一个 char 可以表示 A、□，但不保证能表示一个完整的 Unicode 字符。例如 U+1F600 □□□□ 的码点超过 U+FFFF，需要两个 char 组成的代理项对。

```
char letter = 'A';
char chinese = '中';
int code = chinese;

System.out.println(code); // 20013，即十六进制的 0x4E2D
```

IDE 中显示的文件编码（例如 UTF-8）和 Java 变量的 char 类型属于不同层面，不要把二者混为一谈。

示例：输出基本 CJK 汉字区间

下面遍历 U+4E00～U+9FFF。这只是 Unicode 中的基本 CJK 统一表意文字区，并不包含全部汉字。

```
public class Ex201 {
    public static void main(String[] args) {
        int count = 0;

        for (int iCode = 0x4E00; iCode <= 0x9FFF; iCode++) {
            char cCode = (char) iCode;
            System.out.print(cCode + " ");
            count++;

            if (count % 20 == 0) {
                System.out.println();
            }
        }

        if (count % 20 != 0) {
            System.out.println();
        }
    }
}
```

- 0x 表示十六进制；0x4E00 等于十进制 19968。
- (char) 把整数强制转换为 char，这里的区间可由单个 char 表示。
- print 输出后不换行；println 输出后换行。
- % 是求余运算符，count % 20 == 0 表示“数量是 20 的整数倍”。具体运算规则在 Class 3 展开。

2.1.2 标识符

Java 程序中由程序员命名的名称叫作标识符，例如类名、方法名、变量名和常量名。

命名规则：决定能否通过编译

1. 常见组成字符是字母、数字、下划线 _ 和美元符号 $；Java 也支持许多 Unicode 字符。
2. 第一个字符不能是数字。
3. 标识符区分大小写，例如 age 和 Age 是两个不同的名称。
4. 不能包含空格、-、.、@、中文标点等不允许的符号。
5. 不能与保留关键字同名，也不能使用 true、false、null。
6. Java 9 起，单独的 _ 不能作为普通标识符；_status 仍然合法。

命名规范：让代码容易阅读

命名对象	常用规范	示例
类名	大驼峰，每个单词首字母大写	StudentInfo
变量名、方法名	小驼峰，首词小写	studentAge、getName
常量名	全部大写，单词间用下划线连接	MAX_COUNT
包名	通常全部小写	com.example.demo

中文、$ 或下划线开头的名称可能合法，但实际开发优先使用有意义的英文名称。规则决定“能不能写”，规范决定“写得好不好”。

判断标识符是否合法

名称	合法？	原因
name	是	由字母组成
1_2_3	否	以数字开头
fee$	是	$ 可以出现在标识符中
_status	是	下划线开头，且不是单独的 _
@zhang	否	包含 @
□□	是	Java 支持中文标识符
-data	否	包含 -
china.com	否	包含 .，不能作为一个标识符
□□□	否	包含中文逗号；□、□ 单独使用则合法
while	否	是关键字
hao123	是	字母开头，后面可以有数字

2.1.3 关键字、上下文关键字与字面量

- 标识符：名称由程序员选择，意义由程序员赋予。
- 关键字：由 Java 语言预先规定，具有固定含义。
- 字面量：直接写在程序中表示某个值的记号。

```
int age = 18;
// int：关键字
// age：标识符
// 18：整数字面量
```

常见关键字：

- 基本类型：byte、short、int、long、float、double、char、boolean；
- 类与修饰符：class、public、private、static、final；
- 流程控制：if、else、switch、for、while、break、return。

需要注意：

- const、goto 是保留但未使用的关键字。
- true、false 是布尔字面量，null 是空引用字面量；它们不是关键字，但也不能作为标识符。
- String、System、main 不是关键字。
- var、record 等属于上下文关键字，是否具有特殊含义取决于所在位置。Class 3 会学习 var。

2.2 常量、变量与数据类型

2.2.1 变量、常量与字面量

变量

变量是具有类型和名称的存储位置，用来保存程序运行时使用的数据。理解一个变量时，关注四点：名称、类型、当前值和作用域。

变量必须先声明，后使用。三种基本写法如下：

操作	格式	示例
声明变量	□□□□ □□□;	int age;
给变量赋值	□□□ = □□□;	age = 18;
声明并初始化	□□□□ □□□ = □□□;	int age = 18;

```
int age;            // 声明
age = 18;           // 第一次赋值
age = 19;           // 再次赋值，旧值被替换
System.out.println(age); // 输出 19

int x, y, z;        // 一次声明多个同类型变量
int a = 1, b = 2;   // 分别初始化
```

注意：

- int x, y, z = 0; 只给 z 初始化为 0，没有给 x、y 赋值。
- 赋入变量的值必须与变量类型兼容。
- 不能在同一局部作用域内重复声明同名变量。
- 变量超出自己的作用域后不能继续访问。

变量的分类

按声明位置和用途，可以先认识以下几类：

类别	声明位置	特点
局部变量	方法体或代码块内	使用前必须明确赋值，没有自动默认值
形参	方法或构造方法的参数列表中	调用方法时获得实参传入的值
实例变量	类体内、方法外，没有 static	每个对象各有一份
类变量	类体内、方法外，使用 static	整个类共享一份

实例变量和类变量统称为字段或成员变量。字段和数组元素有默认值，局部变量没有。

类型类别	字段或数组元素的默认值
整数型、char	0 或 \u0000
浮点型	0.0
boolean	false

类型类别	字段或数组元素的默认值
引用类型	null

```
public class VariableDemo {
    int fieldNumber; // 字段有默认值 0

    public static void main(String[] args) {
        int localNumber;
        // System.out.println(localNumber); // 错误：局部变量尚未赋值

        localNumber = 10;
        System.out.println(localNumber);   // 正确
    }
}
```

final 变量与常量

用 final 修饰的变量只能赋值一次：

```
final int MAX_COUNT = 20;
final double PI = 3.1415926;

// MAX_COUNT = 30; // 错误：不能再次赋值
```

常量名通常全部大写，单词间使用下划线。入门阶段可以把 final 基本类型变量理解为“赋值后不能更改的量”。严格来说，不是所有 final 变量都是编译期常量；final 修饰引用时，限制的是引用不能改指向，不代表对象内容完全不可变。

字面量

直接写在源代码中表示固定值的数据叫作字面量，课堂上也常称“直接量”或“字面值常量”。

类别	示例	注意事项
整数字面量	10、0b1010、0xFF、3000000000L	默认通常是 int；long 常用 L 后缀
浮点字面量	3.14、2.5F、1.2E3	默认是 double；float 需要 F 后缀
字符字面量	'A'、'□'、'\n'	使用单引号，只表示一个 UTF-16 代码单元
字符串字面量	"Hello"、""	使用双引号，String 是引用类型
布尔字面量	true、false	不加引号
空引用字面量	null	可赋给引用类型，不能赋给基本类型

-20 严格来说是负号运算符和整数字面量 20 组成的表达式。入门时理解为负整数的写法即可。

2.2.2 八种基本数据类型

Java 的类型分为基本类型和引用类型。基本类型一共八种；String、数组和普通类类型都属于引用类型，void 也不是基本类型。

基本类型速查表

分类	类型	位数	取值范围或特点	示例
整数	byte	8	-128～127	byte level = 10;
整数	short	16	-32768～32767	short year = 2026;
整数	int	32	-2^{31}～$2^{31}-1$，约 ±21 亿	int count = 100;
整数	long	64	-2^{3}～$2^{3}-1$	long population = 8_000_000_000L;
浮点	float	32	IEEE 754 单精度，约 6～7 位十进制有效数字	float height = 1.75F;
浮点	double	64	IEEE 754 双精度，约 15～16 位十进制有效数字	double weight = 65.5;
字符	char	16	0～65535，即 \u0000～\uFFFF	char grade = 'A';
布尔	boolean	语言规范不规定统一存储大小	只能是 true 或 false	boolean passed = true;

位数描述的是类型的值表示，不等于一个变量在 JVM 中必然独占的全部内存。特别是 boolean，不要简单背成“固定占 1 位”或“固定占 1 字节”。

整数类型

Java 整数可以使用不同进制表示：

进制	前缀	示例	十进制值
二进制	0b 或 0B	0b1010	10
八进制	0	012	10
十进制	无	10	10
十六进制	0x 或 0X	0x0A	10

数字之间可以使用下划线增强可读性，例如 1_000_000、8_000_000_000L。下划线不能放在数字开头、结尾、小数点旁边或类型后缀旁边。

整数使用要点：

- 一般整数优先使用 int。
- 超出 int 范围时使用 long，字面量末尾通常写大写 L。
- byte、short 常用于范围明确的数据或大型数组；普通计算中不必为了“省几个字节”强行使用。
- 给 byte、short 赋字面量时，值必须在相应范围内。

```
int count = 100;
long distance = 3_000_000_000L;
byte temperature = -10;

// long wrong = 3_000_000_000; // 错误：字面量本身已超出 int 范围
// byte level = 128;        // 错误：超出 byte 范围
```

浮点类型

浮点数常见写法：

- 十进制形式：3.14；
- 科学计数法：1.2E3，等于 1200.0；
- float 后缀：3.14F；
- double 后缀：3.14D，通常可以省略。

```
double price = 19.9;
float height = 1.75F;
double distance = 1.2E3;
```

float 和 double 使用二进制浮点表示，不能精确表示所有十进制小数：

```
System.out.println(0.1 + 0.2); // 通常显示 0.30000000000000004
```

这不是简单的“保留几位小数”问题，而是二进制表示的精度限制。涉及精确金额计算时，后续会学习 BigDecimal；当前先避免直接用 == 判断两个计算所得浮点数是否完全相等。

字符类型

字符字面量使用英文单引号：

```
char letter = 'A';
char chinese = '中';
char digit = '1'; // 字符 '1'，不是整数 1
```

一些字符需要使用转义序列：

转义序列	含义
\n	换行
\t	水平制表符（Tab）
\r	回车
\b	退格
\f	换页
\'	单引号
\"	双引号
\\	反斜杠
\uXXXX	用四位十六进制数表示一个 UTF-16 代码单元

```
char newline = '\n';
char quote = '\"';
char backslash = '\\';
char unicodeA = '\u0041'; // A
```

常见错误：

```
// char empty = '';   // 错误：字符字面量不能为空
// char word = 'AB';  // 错误：包含两个普通字符
// char text = "A";   // 错误："A" 是 String
```

布尔类型

boolean 只有 true 和 false 两个值，用来表示条件成立或不成立：

```
boolean isFinished = false;
boolean isStudent = true;
```

Java 中 boolean 不能使用 0、1 代替，也不能与数值类型直接相互转换。

基本类型与引用类型的初步区别

```
int age = 18;         // int 是基本类型
char grade = 'A';     // char 是基本类型
String name = "小明";  // String 是引用类型
int[] scores = {90, 85}; // 数组是引用类型
```

当前阶段先记住：

- 基本类型变量直接保存基本值；
- 引用类型变量保存对对象或数组的引用；
- null 表示“不指向任何对象”，只能赋给引用类型；
- 字符使用单引号，字符串使用双引号。

如何选择基本类型

- 普通整数：优先选择 int；范围可能超过约 21 亿时选择 long。
- 普通小数：优先选择 double；明确需要单精度时选择 float。
- 单个 UTF-16 代码单元：选择 char；一段文本选择 String。
- 真或假状态：选择 boolean。
- byte、short 通常在明确的协议、文件格式或大型数组场景中使用。

综合示例

```
public class StudentData {
    public static void main(String[] args) {
        final String SCHOOL_NAME = "示例中学";

        String studentName = "小明";
        int age = 18;
        double score = 89.5;
        char grade = 'B';
        boolean enrolled = true;

        System.out.println("学校：" + SCHOOL_NAME);
        System.out.println("姓名：" + studentName);
        System.out.println("年龄：" + age);
        System.out.println("成绩：" + score);
        System.out.println("等级：" + grade);
        System.out.println("是否在读：" + enrolled);
    }
}
```

输出：

```
学校：示例中学
姓名：小明
年龄：18
成绩：89.5
等级：B
是否在读：true
```

这个示例只负责“定义并保存数据”。下一课会继续学习：如何从键盘输入这些数据，如何通过运算得到新的结果，以及不同基本类型在运算时怎样转换。

常见易错点

1. 标识符不能以数字开头，也不能使用关键字、true、false 或 null。
2. 单独的 _ 从 Java 9 起不能作为普通标识符。
3. int x, y, z = 0; 只初始化了 z。
4. 局部变量没有默认值，读取前必须明确赋值；字段和数组元素才有默认值。
5. final 变量赋值后不能再次赋值，但 final 引用所指对象的内容不一定不可变。
6. 整数字面量默认通常是 int，较大的 long 字面量要加 L。
7. 浮点字面量默认是 double，赋给 float 时要加 F。
8. char 使用单引号，String 使用双引号；String 不是基本类型。
9. 一个 char 不一定等于一个完整的 Unicode 字符。
10. boolean 只能是 true 或 false，不能使用 0 或 1 代替。
11. null 只能赋给引用类型，不能赋给八种基本类型。
12. 浮点数不能精确表示所有十进制小数。

课后自测

1. studentName、2name、class、_name、true 中，哪些可以作为变量名？
2. int a, b = 10; 之后，能直接读取局部变量 a 的值吗？
3. 八种基本数据类型分别是什么？String 属于哪一类？
4. float price = 3.14; 为什么不能通过编译？怎样修改？
5. 3000000000 为什么通常要写成 3000000000L？
6. 'A' 和 "A" 分别是什么类型？
7. char 的范围是多少？一个 char 一定能表示一个完整字符吗？
8. 局部变量与字段的默认值规则有什么区别？
9. final int LIMIT = 10; 之后还能写 LIMIT = 20; 吗？
10. 为什么 0.1 + 0.2 可能不精确等于十进制的 0.3？

参考答案

1. studentName、_name 合法；其余分别是数字开头、关键字和布尔字面量。
2. 不能。只初始化了 b，局部变量 a 在读取前必须明确赋值。
3. byte、short、int、long、float、double、char、boolean；String 是引用类型。
4. 3.14 默认是 double；可写 float price = 3.14F;，或改为 double price = 3.14;。
5. 它超出 int 范围；L 让该字面量成为 long。
6. 'A' 是 char，"A" 是 String。
7. \u0000～\uFFFF；不一定，部分 Unicode 字符需要两个 char。
8. 局部变量没有默认值，使用前必须赋值；字段和数组元素有规定的默认值。
9. 不能，final 变量只能赋值一次。
10. 因为 float、double 使用有限位数的二进制浮点表示，某些十进制小数无法被精确表示。

本课小结与下一课衔接

- 标识符是程序员定义的名称，关键字由 Java 语言规定，字面量直接表示值。
- 变量有确定的类型、名称、值和作用域；局部变量必须在读取前明确赋值。
- final 用于限制变量只能赋值一次，常量名通常采用全大写加下划线。
- Java 有八种基本数据类型；String 和数组属于引用类型。
- 整数和浮点字面量有默认类型，必要时使用 L、F 后缀。
- char 是 UTF-16 代码单元，不保证一个 char 就是一个完整字符。

完成本课后，已经知道“数据是什么、使用什么类型保存”。Class 3 将按以下顺序继续：

1. 2.2.3：使用 Scanner 输入数据，以及 print、println、printf 输出数据；
2. 2.2.4：使用 var 推断局部变量类型；
3. 3.1：使用算术、关系、逻辑、赋值、条件和位运算符处理数据；
4. 3.3：理解运算时的类型提升、自动转换与强制转换。